



Rapport de Projet ML : Identification acoustique d'espèces dans le Pantanal sans réseaux de neurones

BirdCLEF+ 2026

Yuxiang ZHANG – Ziyue ZHANG

Projet de Machine Learning

22 mai 2026

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Problématique et hypothèses	2
2	Analyse des données	2
2.1	Jeu de données	2
2.2	Répartition par classe biologique et distribution en longue traîne	3
2.3	Couverture des espèces	3
2.4	Données non audio	3
3	Méthode proposée	4
3.1	Pipeline général	4
3.2	Prétraitement audio et sélection de fenêtre	4
3.3	Ingénierie des caractéristiques	4
3.3.1	Correction cruciale des descripteurs spectraux	5
3.4	Classifieurs utilisés	5
3.5	Techniques écartées lors d'une version antérieure	5
4	Démarche expérimentale	6
4.1	Division entraînement / validation	6
4.2	Métrique d'évaluation	6
4.3	Déroulement de l'entraînement	6
4.4	Difficultés techniques rencontrées	6
5	Résultats	6
5.1	Comparaison des classifieurs sur la validation interne	6
5.2	Décalage de distribution : le résultat central	7
5.3	Réduction du décalage par ajout de soundscapes	8
5.4	Score Kaggle final	9
5.5	Analyse critique des erreurs	9
5.5.1	Échec de l'approche hiérarchique	9
5.5.2	Échec de la calibration des probabilités	9
5.5.3	Cas d'erreur spécifique	10
5.5.4	Analyse par strates	10
5.6	Analyse des hyperparamètres	11
5.7	Importance des caractéristiques	11
6	Limites et perspectives	12
7	Conclusion	12

Résumé

Ce rapport présente une solution de *machine learning* classique pour la compétition BirdCLEF+ 2026. L’objectif est d’identifier automatiquement 234 espèces animales à partir d’enregistrements audio collectés dans le Pantanal brésilien, sans utiliser de réseaux de neurones. Notre pipeline combine des descripteurs acoustiques artisanaux, une représentation par *Bag of Audio Words* avec *bagging* et une factorisation NMF, suivi de l’entraînement de sept classifieurs. Sur l’ensemble de validation interne, le modèle LightGBM atteint un AUC ROC macro de 0,8591. Toutefois, l’évaluation sur les soundscapes de terrain montre une chute à 0,5222. L’ajout contrôlé de soundscapes à l’entraînement améliore l’AUC en distribution réelle de +0,086, et le score public Kaggle final atteint **0,744**, validant le diagnostic de décalage de distribution. Nous analysons en détail ce phénomène ainsi que les erreurs résiduelles, et proposons des pistes d’amélioration. Le code développé respecte strictement l’interdiction des réseaux de neurones et les contraintes de la compétition.

1 Introduction

1.1 Contexte

La surveillance de la biodiversité dans les zones reculées comme le Pantanal repose de plus en plus sur l’acoustique passive. La compétition BirdCLEF+ 2026 vise à identifier automatiquement 234 espèces animales à partir de segments audio de 5 secondes, enregistrés en continu par des capteurs passifs. Le défi est rendu plus ardu par la diversité des classes biologiques présentes, une distribution des espèces en longue traîne, et la présence d’espèces totalement absentes de l’ensemble d’entraînement.

La contrainte pédagogique du projet est l’interdiction d’utiliser des réseaux de neurones. Nous nous restreignons donc aux méthodes d’apprentissage automatique classiques. Cette contrainte place notre approche à contre-courant des solutions dominantes de l’état de l’art, qui exploitent presque exclusivement des réseaux de neurones convolutifs pré-entraînés sur spectrogrammes.

1.2 Problématique et hypothèses

Le problème est structuré par trois difficultés majeures :

1. **Distribution en longue traîne** : les espèces sont très inégalement représentées avec médiane de 125 enregistrements, 27 espèces ayant 10 exemples ou moins, 4 espèces avec un seul exemple.
2. **Espèces « zero-shot »** : 28 des 234 espèces à prédire n’ont aucun exemple dans l’ensemble d’entraînement.
3. **Décalage de distribution** : les données d’entraînement proviennent d’enregistrements iNaturalist (proches, propres, mono-espèce), tandis que le jeu de test est constitué de *soundscapes* de terrain (bruités, lointains, multi-espèces).

Notre hypothèse principale est que des descripteurs acoustiques bien conçus, associés à des classifieurs robustes, peuvent atteindre une bonne performance en distribution, mais subiront une perte significative lors du passage à la distribution de terrain. Une grande partie de notre analyse est consacrée à quantifier et comprendre ce décalage.

2 Analyse des données

2.1 Jeu de données

L’ensemble d’entraînement comprend 35 549 enregistrements audio au format `.ogg`, accompagnés de métadonnées. Chaque échantillon est associé à une étiquette primaire et parfois à des étiquettes secondaires. On dénombre 206 espèces distinctes dans les étiquettes primaires, mais le

fichier de soumission requiert la prédiction de 234 espèces, 28 espèces sont donc absentes de l’entraînement.

2.2 Répartition par classe biologique et distribution en longue traîne

La Figure 1 illustre les deux principaux déséquilibres du jeu de données.

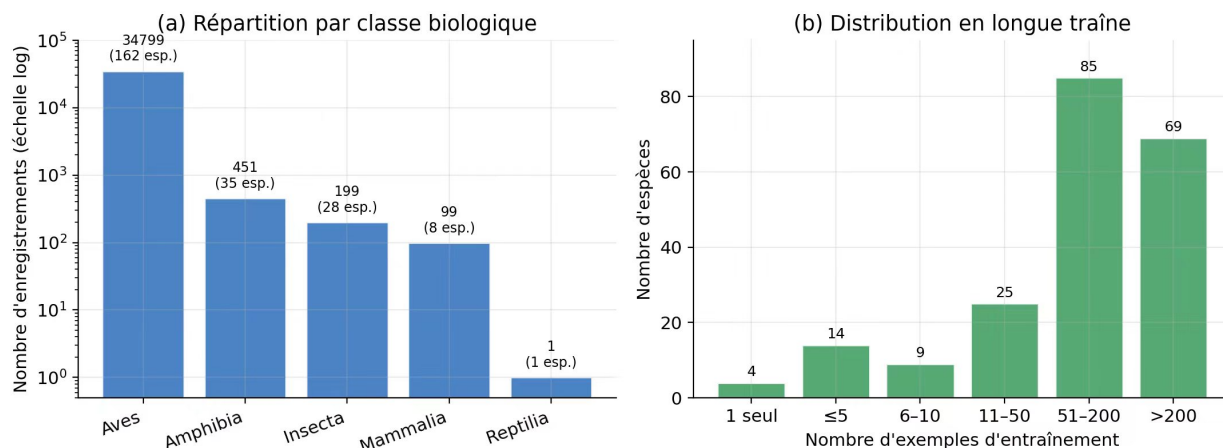


FIGURE 1 – (a) Répartition des enregistrements par classe biologique — les oiseaux dominent très largement. (b) Distribution du nombre d’espèces par strate d’effectif — une longue traîne avec 27 espèces ayant ≤ 10 exemples et 4 espèces avec un seul exemple.

- **Par classe biologique** : les oiseaux totalisent 34 799 enregistrements (162 espèces), suivis des amphibiens (451 enregistrements, 35 espèces), des insectes (199 enregistrements, 28 espèces), des mammifères (99 enregistrements, 8 espèces) et des reptiles (1 seul enregistrement). Les oiseaux représentent 98 % du volume total, créant un fort déséquilibre inter-classes.
- **Par fréquence d’espèce** : 69 espèces possèdent plus de 200 exemples, mais 27 espèces en ont 10 ou moins, dont 4 avec un seul. La médiane est de 125 enregistrements par espèce.

2.3 Couverture des espèces

- Espèces présentes dans l’entraînement : **206 / 234**.
- Espèces « zero-shot » : **28**.

AUC ROC macro comme métrique officielle ignore les espèces sans aucun exemple positif dans le jeu de test. Nous avons donc choisi de prédire systématiquement 0 pour les 28 espèces zero-shot : si elles n’apparaissent pas dans le test, elles n’affectent pas le score ; si elles apparaissent, nous ne disposons de toute façon d’aucune donnée pour les apprendre.

2.4 Données non audio

Le fichier `train.csv` contient des métadonnées non audio telles que la localisation géographique et une évaluation de qualité. Ces informations pourraient théoriquement améliorer la classification parce que certaines espèces étant géographiquement localisées, mais le jeu de test ne fournit pas ces champs au niveau de chaque fenêtre.

Qualité d’enregistrement (rating). 64 % des enregistrements possèdent une note de qualité concentrée entre 4,0 et 5,0 ; les 36 % restants ne sont pas notés. Cet attribut pourrait servir à pondérer les exemples à l’entraînement, mais l’absence de note pour plus d’un tiers des données en limite l’usage direct.

Localisation (latitude, longitude). L’analyse géographique révèle un constat important. Les données d’entraînement sont **réparties sur l’ensemble du globe** (latitude de $-54,9$ à $69,6$, longitude de $-159,7$ à $175,3$), alors que le jeu de test provient exclusivement du **Pantanal**. De plus, les classes biologiques occupent des zones distinctes, et certaines espèces sont géographiquement très concentrées tandis que d’autres sont cosmopolites .

Ces observations montrent que la géolocalisation possède un réel pouvoir discriminant et constituerait un *a priori* contextuel puissant, un modèle pourrait apprendre que certaines espèces sont improbables dans le Pantanal. Plus encore, ce décalage géographique constitue une **seconde dimension du décalage de distribution**, complémentaire au décalage acoustique identifié en section 5.3.

3 Méthode proposée

3.1 Pipeline général

Notre pipeline suit le schéma classique d’apprentissage automatique : prétraitement audio, extraction de caractéristiques, entraînement de classifieurs binaires par espèce, et fusion optionnelle des prédictions. L’accent a été mis sur la qualité et la diversité des descripteurs acoustiques ainsi que sur la robustesse des modèles face au déséquilibre des classes.

3.2 Prétraitement audio et sélection de fenêtre

Tous les fichiers audio sont ré-échantillonnés à 32 000 Hz en mono. Pour chaque enregistrement, nous sélectionnons la **fenêtre de 5 secondes possédant l’énergie RMS maximale**, qui correspond généralement à un chant ou cri actif de l’animal. Cette approche améliore le rapport signal-sur-bruit par rapport à un découpage central arbitraire. Si le fichier est plus court que 5 secondes, il est répété. Un spectrogramme de Mel de 128 bandes, $n_{fft} = 2048$, $hop_length = 512$, $f_{min} = 20$ Hz, $f_{max} = 16\,000$ Hz est ensuite calculé en échelle logarithmique pour chaque segment.

3.3 Ingénierie des caractéristiques

Le vecteur de caractéristiques final comporte **2086 dimensions**, résultant de la concaténation de trois blocs :

1. **Descripteurs *handcrafted*** (1 126 dimensions) :
 - Statistiques segmentées du log-mel : le spectrogramme est découpé en 3 segments temporels, pour chacun desquels on calcule moyenne et écart-type des 128 bandes Mel.
 - MFCC + Δ + Δ^2 : 20 coefficients MFCC, leurs dérivées première et seconde, résumés chacun par moyenne, écart-type, asymétrie et aplatissement.
 - Chroma : énergie sur les 12 classes de hauteur.
 - Contraste spectral : différence pic-vallée sur 7 sous-bandes.
 - Rolloff spectral, bande passante, RMS, taux de passage par zéro : résumés par moyenne, écart-type, asymétrie et aplatissement.
 - Enveloppe temporelle et modulation : statistiques de l’enveloppe RMS, 40 coefficients d’autocorrélation, et 20 coefficients du spectre de modulation.
2. **Bag of Audio Words avec *bagging*** (768 dimensions) : trois codebooks de 256 mots sont construits par K-Means sur des échantillons bootstrap distincts de trames Mel. Chaque codebook produit un histogramme normalisé des occurrences des mots ; les trois histogrammes sont concaténés. Cette technique capture la distribution des motifs spectraux locaux de manière robuste.
3. **Dictionnaire NMF** (192 dimensions) : une factorisation non-négative de 64 composantes est apprise sur les trames Mel normalisées dans $[0, 1]$. Pour chaque composante, on agrège

la moyenne, l'écart-type et le maximum de ses activations temporelles. La NMF fournit une représentation douce, *part-based*, complémentaire au BoAW.

Aucune réduction de dimension comme PCA n'est appliquée : les modèles arborescents gèrent naturellement les données de grande dimension et ces étapes ont montré expérimentalement qu'elles dégradaient les performances.

3.3.1 Correction cruciale des descripteurs spectraux

Lors du développement, nous avons constaté que les descripteurs `chroma_stft`, `spectral_contrast`, `spectral_rolloff` et `spectral_bandwidth` étaient calculés en passant le spectrogramme log-mel aux fonctions de `librosa`, alors que celles-ci attendent un spectrogramme STFT à échelle linéaire. Aucune erreur n'était levée, mais les valeurs obtenues étaient physiquement incohérentes et leur pouvoir discriminant quasi nul. Le calcul correct, utilisant la magnitude de la STFT linéaire, a été rétabli, ce qui a constitué l'une des améliorations les plus significatives du pipeline.

3.4 Classifieurs utilisés

Nous avons entraîné sept classifieurs couvrant cinq familles méthodologiques, tous dans une configuration *Binary Relevance*. Cette approche permet de gérer séparément le déséquilibre propre à chaque espèce et de paralléliser les calculs.

- **LightGBM** : boosting de gradient sur arbres. Rapide, performant en haute dimension, il gère le déséquilibre via le paramètre `scale_pos_weight` (rapport négatifs/positifs). Ses hyperparamètres ont été choisis pour équilibrer capacité de modélisation et régularisation.
- **Random Forest** : 100 arbres, profondeur maximale 12, `class_weight='balanced'`. Il produit des probabilités plus lisses et est très robuste au bruit.
- **Extra-Trees** : variante encore plus aléatoire du Random Forest, introduisant une diversification supplémentaire.
- **HistGradientBoosting** : version histogramme du gradient boosting, proche de LightGBM, avec l'avantage d'accepter nativement les valeurs manquantes.
- **Régression logistique** : modèle linéaire probabiliste, simple et rapide, permettant d'évaluer la séparabilité linéaire des données.
- **Gaussian Naive Bayes** : modèle génératif bayésien, utilisé comme référence simple.
- **Nyström + SGD** : approximation du noyau RBF via la méthode de Nyström avec 300 composantes et $\gamma = 10^{-3}$, suivie d'une régression logistique entraînée par SGD. Cette approche capture des frontières non-linéaires de manière scalable et remplace avantageusement un SVM linéaire qui ne convergait pas sur nos données.

Chaque classifieur est entraîné uniquement pour les espèces disposant d'au moins 3 exemples positifs (10 pour les modèles linéaires et à noyau). Les espèces trop rares ou absentes sont prédites à 0.

3.5 Techniques écartées lors d'une version antérieure

Dans une version antérieure du pipeline, nous avons testé une classification hiérarchique par pseudo-ordres acoustiques (clustering K-Means des espèces) et un calibrage adaptatif des probabilités (isotonique / Platt). Ces deux techniques ont systématiquement dégradé l'AUC macro (de 0,80 à 0,73 pour la hiérarchie, de 0,80 à 0,75 pour la calibration) et ont donc été retirées du pipeline final. Leur échec souligne la difficulté d'introduire une structure *a priori* lorsque les données sont fortement déséquilibrées et que la taxonomie acoustique ne coïncide pas avec la biologie.

4 Démarche expérimentale

4.1 Division entraînement / validation

Nous avons utilisé un split stratifié « rare-aware » : chaque espèce possédant plus d'un exemple contribue à hauteur de 20 % à l'ensemble de validation ; les 4 espèces à un seul exemple restent intégralement dans l'entraînement. Les indices de division sont sauvegardés pour assurer la reproductibilité.

4.2 Métrique d'évaluation

La métrique officielle de la compétition est l'**AUC ROC macro** calculée uniquement sur les classes ayant au moins un exemple positif dans le jeu de test. Nous reproduisons exactement cette métrique sur notre ensemble de validation interne .

4.3 Déroulement de l'entraînement

Le pipeline complet est exécuté sur CPU sans accès Internet. L'extraction parallélisée des caractéristiques et l'utilisation d'un cache disque réduisent considérablement les temps de calcul. L'entraînement des modèles LightGBM et Random Forest bénéficie du multi-threading interne, permettant de traiter les 234 classifieurs en environ 1,5 heure. Le temps total d'exécution est d'environ 2 heures.

4.4 Difficultés techniques rencontrées

Plusieurs obstacles d'ingénierie ont dû être surmontés :

- **Conflit OpenMP sous Windows** : la présence simultanée d'Intel MKL et de LLVM OpenMP provoquait des crashes silencieux de scikit-learn. La solution a consisté à basculer l'ensemble de la pile BLAS vers OpenBLAS.
- **Non-convergence du SVM linéaire** : le solveur liblinear de `LinearSVC` ne convergeait pas sur la matrice complète ($28\,510 \times 2\,086$), produisant des AUC proches de 0,49. Le remplacement par l'approximation de Nyström + SGD a résolu le problème en quelques minutes.
- **Déploiement Kaggle** : la soumission a nécessité la suppression des dossiers `__pycache__`, la recreation des modules `src.*` pour le dépicklage, et la désactivation de l'accès Internet. Le notebook d'inférence ré-implémente en pur NumPy les classes BoAW et NMF pour éviter toute dépendance à scikit-learn lors de la prédiction, et limite le nombre d'itérations NMF à 30. La prédiction LightGBM est parallélisée avec `num_threads=4`.

5 Résultats

5.1 Comparaison des classifieurs sur la validation interne

Le Tableau 1 et la Figure 2 présentent les AUC macro obtenues sur l'ensemble de validation interne.

Classifieur	Famille	AUC macro
LightGBM	Boosting gradient	0,8591
ExtraTrees	Bagging randomisé	0,8351
Random Forest	Bagging	0,8301
Nyström + SGD	Méthode à noyau	0,8119
Régression logistique	Linéaire	0,8066
HistGradientBoosting	Boosting histogramme	0,7668
Naive Bayes gaussien	Bayésien	0,6012
Rank-Averaging (LGB+RF)	Ensemble	0,8587
Moyenne pondérée (LGB+RF)	Ensemble	0,8592

TABLE 1 – AUC ROC macro sur la validation interne de 7 109 échantillons.

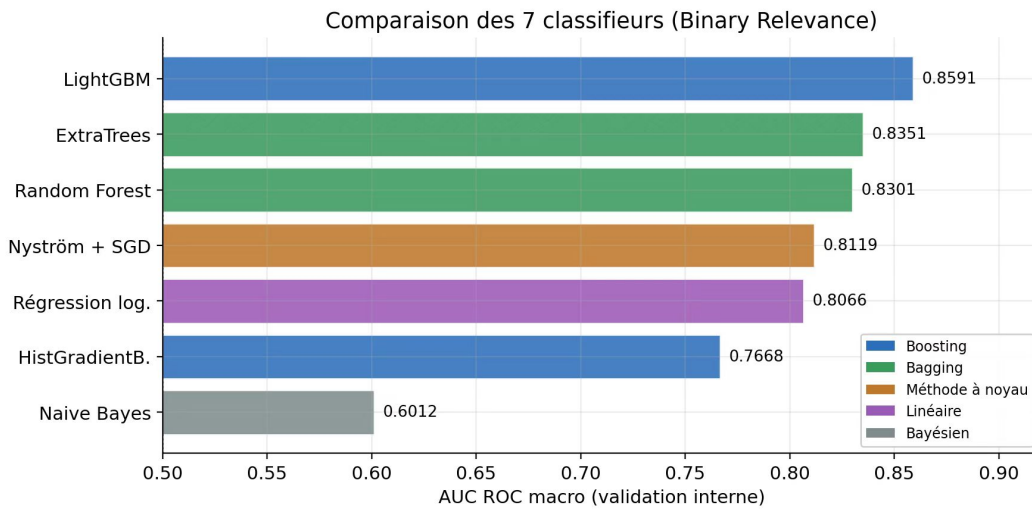


FIGURE 2 – Comparaison des sept classifieurs sur la validation interne. LightGBM domine avec 0,8591, suivi des méthodes arborescentes.

LightGBM domine nettement avec 0,8591, suivi par les autres méthodes arborescentes. Les modèles linéaires et bayésiens sont nettement en retrait, ce qui est cohérent avec la complexité non-linéaire du problème. L'ensemble par rang atteint 0,8587, soit un score très proche du meilleur modèle individuel, indiquant que la fusion n'apporte pas de gain significatif lorsque les modèles ne sont pas suffisamment diversifiés. Pour la soumission, nous avons donc retenu le modèle LightGBM seul, qui allie simplicité et performance maximale.

5.2 Décalage de distribution : le résultat central

Nous avons évalué le meilleur modèle LightGBM sur les `train_soundscape`s étiquetés, qui sont des enregistrements de terrain multi-espèces, représentatifs du véritable jeu de test. Le Tableau 2 et la Figure 3 résument la chute de performance et sa correction.

Évaluation	Distribution	AUC macro
Validation interne	iNaturalist (entraînement)	0,8591
<code>train_soundscape</code> s	Terrain (multi-espèces)	0,5222

TABLE 2 – Impact du décalage de distribution sur l'AUC macro.

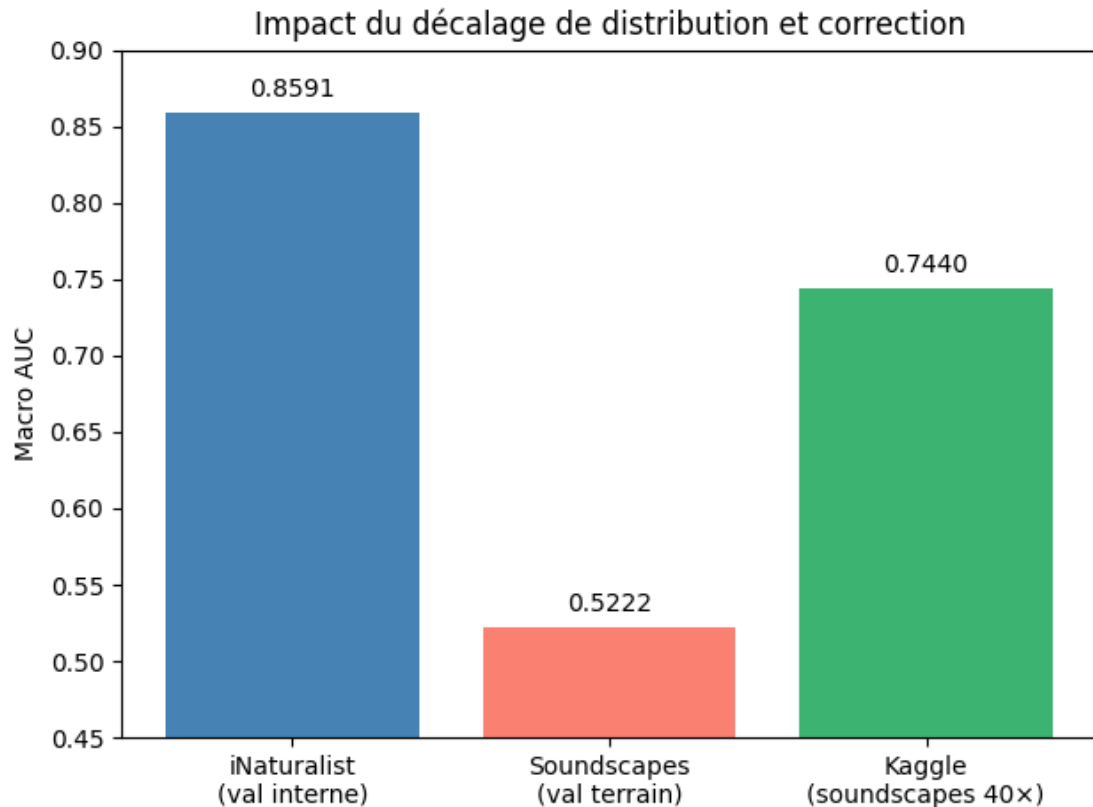


FIGURE 3 – Comparaison de l’AUC macro sur trois distributions : validation interne iNaturalist (0,8591), soundscapes de terrain (0,5222), et score Kaggle final après ajout des soundscapes (0,744).

L’AUC s’effondre de 0,86 à 0,52. Ce résultat est le plus important du projet : il confirme que le modèle a appris les caractéristiques d’enregistrements propres et mono-espèce, et généralise très difficilement aux soundscapes bruités, lointains et multi-espèces.

5.3 Réduction du décalage par ajout de soundscapes

Le diagnostic précédent suggère une piste directe : entraîner le modèle sur des données de la distribution cible. Les `train_soundscapes` étiquetés constituent précisément de telles données ; leur usage à l’entraînement est seulement autorisé par la compétition. Cependant, ces 1 478 fenêtres représentent moins de 5 % du volume d’entraînement iNaturalist ; un simple mélange ne suffit donc pas à modifier le comportement du modèle.

Pour compenser ce déséquilibre, nous avons augmenté le poids des fenêtres de soundscapes pendant l’entraînement en les dupliquant. Nous avons testé trois facteurs de duplication : **10×**, **20×** et **40×**. Un facteur trop faible ne modifiait quasiment pas la performance sur la validation terrain ; un facteur trop élevé risquait de dégrader la performance sur les échantillons iNaturalist. Avec 40×, la validation terrain a montré une nette amélioration tout en conservant une bonne performance sur la validation iNaturalist, indiquant que les deux distributions étaient alors à peu près équilibrées.

Après avoir fixé le facteur à 40, nous avons d’abord vérifié que ces données de terrain sont *apprenables* : un LightGBM entraîné uniquement sur des soundscapes atteint une AUC macro de **0,80** dont la médiane est 0,91 sur la validation terrain. La difficulté ne vient donc pas des soundscapes eux-mêmes, mais du décalage de distribution.

Nous avons ensuite mené une **expérience contrôlée** : sur le *même* ensemble de validation de 16 fichiers soundscape réservés, nous comparons un modèle entraîné sur iNaturalist seul à un modèle entraîné sur iNaturalist augmenté des 50 fichiers soundscape d’apprentissage, ces derniers étant pondérés par le facteur 40×. Le Tableau 3 résume le résultat.

Données d'entraînement	AUC macro (validation terrain)
iNaturalist seul	0,6622
iNaturalist + soundscapes (40×)	0,7484
Gain	+0,086

TABLE 3 – Effet de l'ajout du dataset soundscapes pondérés 40× à l'entraînement, mesuré sur le même ensemble de validation terrain.

L'ajout de données de terrain améliore l'AUC en distribution réelle de **+0,086**, un gain net et causalement attribuable (les deux conditions partagent exactement le même ensemble de validation). Ce résultat confirme que le décalage de distribution est la cause principale de la chute de performance. Notre modèle de soumission final est donc entraîné sur l'union d'iNaturalist et de l'ensemble des soundscapes étiquetés.

5.4 Score Kaggle final

La Figure 4 présente les scores publics de nos deux soumissions principales. La Version 6, entraînée uniquement sur les données iNaturalist, a obtenu un score de **0,568**. La Version 8, qui intègre les soundscapes de terrain avec une pondération de 40×, atteint **0,744**. Ce gain de +0,176 confirme l'efficacité de l'ajout des données de la distribution cible pour réduire le décalage de domaine.

Submissions

Select up to 2 submissions that will count towards your final leaderboard score. If less than 2 are selected, Kaggle will automatically select from your best scoring submissions. [Learn More](#)

Auto-selection candidates ?

☒ All
 ☐ Successful
 ☐ Selected
 ☐ Errors
 Recent ▾

Submission and Description	Public Score ⓘ	Select
☒ notebook995f33ecea - Version 8 Succeeded · 24m ago	0.744	☐
☒ notebook995f33ecea - Version 6 Succeeded · 2d ago · 2	0.568	☐

FIGURE 4 – Scores publics Kaggle des deux soumissions : Version 6 (iNaturalist seul) à 0,568 et Version 8 (iNaturalist + soundscapes 40×) à 0,744.

5.5 Analyse critique des erreurs

5.5.1 Échec de l'approche hiérarchique

La classification hiérarchique par pseudo-ordres acoustiques a provoqué une chute d'AUC de 0,80 à 0,73. Le classifieur d'ordre, entraîné sur seulement 202 espèces avec des clusters acoustiques peu fiables, introduisait un bruit préjudiciable aux espèces rares.

5.5.2 Échec de la calibration des probabilités

La calibration adaptative a réduit l'AUC de 0,80 à 0,75. Comme le montre la Figure 5, les courbes de fiabilité se rapprochent de la diagonale pour les espèces fréquentes, mais la calibration dégrade la métrique macro car elle est très instable sur les espèces à faible effectif de validation.

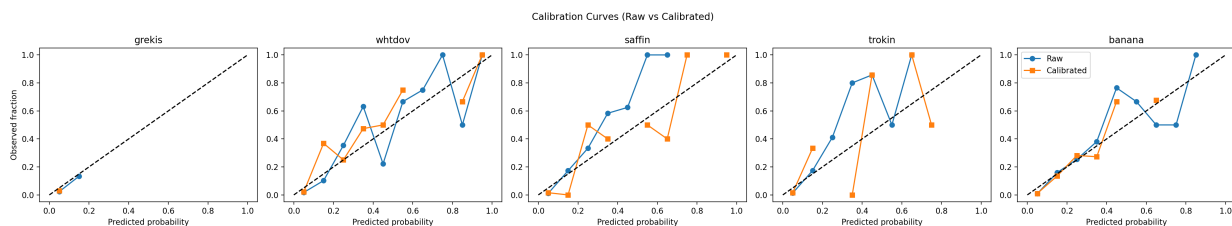


FIGURE 5 – Courbes de calibration avant/après pour les 5 espèces les plus fréquentes.

5.5.3 Cas d’erreur spécifique

Parmi les 75 espèces présentes dans les `train_soundscape`s, 14 atteignent une AUC supérieure à 0,7, mais 22 tombent en dessous de 0,5. L’espèce 66971 est particulièrement frappante : avec 149 positifs, elle n’obtient qu’une AUC de 0,273. Cela suggère que sa signature acoustique diffère radicalement entre iNaturalist et le terrain, ou qu’elle est systématiquement confondue avec d’autres espèces co-occurentes.

5.5.4 Analyse par strates

Pour comprendre où le modèle réussit et échoue, nous décomposons l’AUC de validation (LightGBM, 202 espèces) selon deux axes.

Par classe biologique. Le Tableau 4 montre une forte hétérogénéité. Les oiseaux, massivement représentés, atteignent une AUC moyenne de 0,88. Les amphibiens, malgré 30 espèces évaluées, plafonnent à 0,75 : leurs vocalisations sont intrinsèquement plus difficiles à discriminer. Les valeurs élevées des insectes et mammifères doivent être interprétées avec prudence, le faible nombre d’espèces évaluées (3 et 7) rendant l’estimation peu stable.

Classe	Espèces évaluées	AUC moyenne	Médiane
Insecta	3	0,968	0,984
Aves	162	0,879	0,889
Mammalia	7	0,824	0,932
Amphibia	30	0,750	0,721

TABLE 4 – AUC macro par classe biologique. Les classes à faible effectif d’espèces donnent des estimations peu fiables.

Par fréquence d’entraînement. Le Tableau 5 révèle une relation monotone et nette entre le nombre d’exemples d’entraînement et la performance. La Figure 6 illustre cette tendance.

Strate (exemples d’entraînement)	Espèces	AUC moyenne	Médiane
Très rare (≤ 10)	23	0,744	0,750
Rare (11–50)	25	0,827	0,870
Moyen (51–200)	85	0,874	0,887
Fréquent (> 200)	69	0,891	0,898

TABLE 5 – AUC macro par strate de fréquence d’entraînement (validation interne). La performance croît de façon monotone avec le nombre d’exemples.

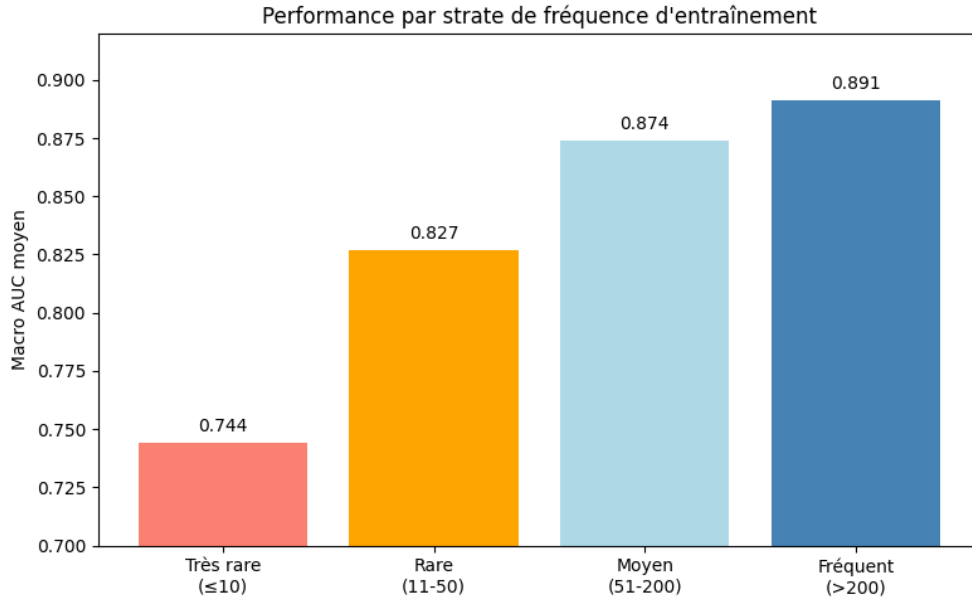


FIGURE 6 – AUC macro moyen par strate de fréquence d’entraînement. L’écart de 0,15 quantifie le coût de la longue traîne.

Les espèces les plus mal classées ($AUC \approx 0,5$) sont presque toutes des amphibiens ou des espèces à très faible effectif, pour lesquelles l’AUC est à la fois statistiquement instable et pénalisée par le manque de données. À l’inverse, les meilleures performances ($AUC > 0,99$) concernent des espèces aux signatures acoustiques très distinctives, indépendamment de leur classe.

5.6 Analyse des hyperparamètres

Pour LightGBM, les hyperparamètres ont été optimisés via Optuna (15 essais bayésiens) sur une espèce représentative. Les valeurs retenues sont : `num_leaves=31`, `learning_rate=0.05`, `min_child_samples=20`, `feature_fraction=0.8`, `bagging_fraction=0.8`, `bagging_freq=5`. Le paramètre `scale_pos_weight` est fixé individuellement pour chaque espèce au ratio négatifs/positifs, ce qui s’est avéré crucial pour les classes à faible effectif. Les autres modèles utilisent des hyperparamètres par défaut qui se sont révélés satisfaisants sans réglage fin supplémentaire.

5.7 Importance des caractéristiques

L’analyse de l’importance des caractéristiques (gain normalisé, moyenné sur les espèces) pour le modèle LightGBM est présentée dans la Figure 7. Les statistiques segmentées du Mel (SegMel) restent le groupe le plus contributif (29,8 % de l’importance totale), suivies par les coefficients MFCC (20,9 %), le BoAW (18,7 %) et la NMF (14,5 %). Les descripteurs corrigés (enveloppe temporelle, contraste spectral, chroma, etc.) apportent également une contribution non négligeable, totalisant environ 16 % de l’importance cumulée. Ce résultat confirme que la correction du calcul des descripteurs spectraux a permis d’exploiter pleinement la diversité des caractéristiques acoustiques, chaque groupe apportant une information complémentaire.

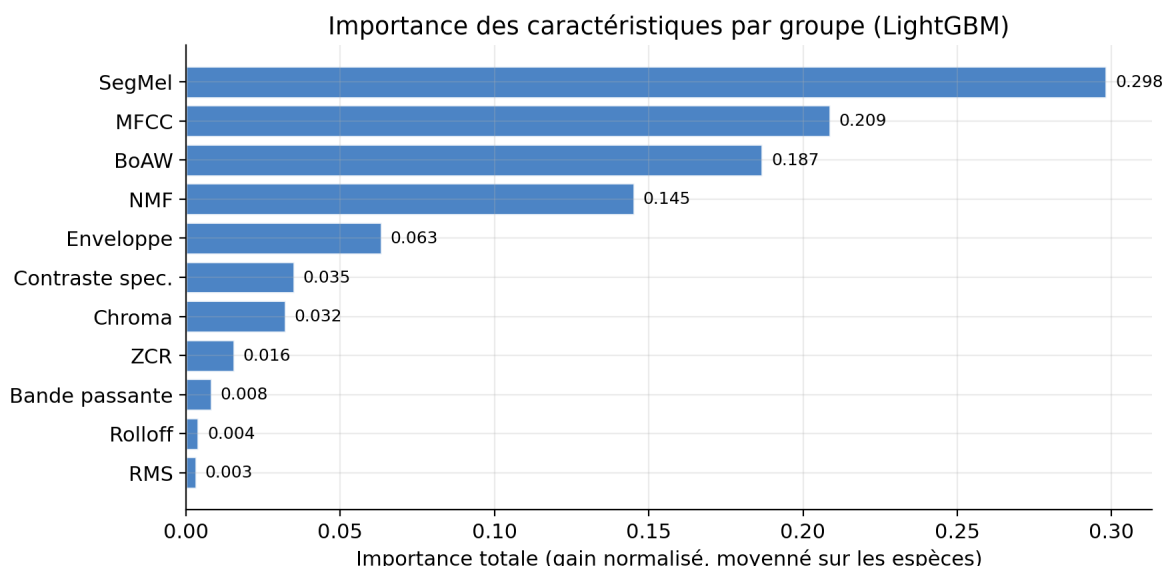


FIGURE 7 – Importance des caractéristiques par groupe pour le modèle LightGBM (gain normalisé, moyenné sur toutes les espèces).

6 Limites et perspectives

- **Réduction supplémentaire du décalage** : au-delà de l’ajout des soundscapes, des augmentations de données simulant le bruit de terrain pourraient réduire davantage l’écart avec le test réel.
- **Apprentissage multi-étiquettes** : les soundscapes contiennent souvent plusieurs espèces simultanément. Un modèle explicitement multi-label comme des chaînes de classifieurs pourrait mieux capturer ces co-occurrences.
- **Espèces zero-shot** : l’intégration de données externes, autorisée par la compétition, permettrait de couvrir les 28 espèces manquantes.
- **Optimisation individuelle des hyperparamètres** : une recherche bayésienne par espèce améliorerait probablement les performances des classes difficiles, au prix d’un temps de calcul conséquent.
- **Données non audio** : les métadonnées géographiques n’ont pas été exploitées. Si le jeu de test les fournissait, elles pourraient constituer un a priori contextuel puissant.
- **Comparaison avec des modèles pré-entraînés** : les solutions de pointe de BirdCLEF utilisent des CNN pré-entraînés (EfficientNet) avec une forte augmentation de données. Notre contrainte « sans réseaux de neurones » explique en grande partie l’écart de performance et illustre pourquoi l’état de l’art a convergé vers l’apprentissage profond pour cette tâche.

7 Conclusion

Nous avons conçu, implémenté et évalué un pipeline complet d’apprentissage automatique classique pour l’identification multi-espèces dans des enregistrements audio. Le modèle LightGBM atteint un AUC macro de 0,8591 sur la validation interne, démontrant la pertinence des descripteurs acoustiques corrigés et de la stratégie *Binary Relevance*. La chute à 0,52 sur les soundscapes de terrain révèle un décalage de distribution majeur, que l’ajout de ces mêmes soundscapes à l’entraînement (pondération 40×) permet de réduire significativement (+0,086). Le score public Kaggle final de **0,744** confirme l’efficacité de cette adaptation au domaine. Ces résultats mettent en lumière les limites de l’approche lorsque les données d’entraînement ne sont pas représentatives du

déploiement, et motivent les nombreuses pistes d'amélioration identifiées. Ce projet nous a permis de mettre en œuvre une large palette de techniques d'apprentissage, de la fouille de caractéristiques à la fusion d'ensemble, tout en développant une démarche expérimentale rigoureuse.

Remerciements

Nous remercions l'équipe pédagogique pour ce projet stimulant et les organisateurs de Bird-CLEF+ pour la mise à disposition des données.

Annexe : structure du code final

Le code est organisé en modules réutilisables et notebooks :

1. `config.py` : constantes et chemins.
2. `audio.py` : chargement audio, sélection de la fenêtre de plus forte énergie.
3. `features.py` : extraction des 1 126 dimensions *handcrafted* (avec correction STFT).
4. `boaw.py` : modèle Bagging-BoAW (3 codebooks de 256 mots, K-Means en numpy pur).
5. `nmf_features.py` : modèle NMF (64 composantes, 192 dimensions).
6. `feature_pipeline.py` : extraction parallèle, cache, apprentissage des modèles non-supervisés.
7. `models.py` : entraînement *Binary Relevance*, *rank-averaging*, évaluation.
8. `train.py` : script principal d'entraînement (iNaturalist seul).
9. `train_extra_models.py` : entraînement des 5 classifieurs supplémentaires (ExtraTrees, HistGB, LogReg, GaussianNB, Nyström+SGD).
10. `train_with_soundscapes.py` : expérience d'ajout de soundscapes (pondération 40×) et évaluation sur validation terrain.
11. `00_explore_data.ipynb` : notebook d'analyse exploratoire et de tests unitaires.
12. `notebook Kaggle d'inférence` : notebook de soumission final. Ré-implémente en pur NumPy les classes BoAW et NMF pour s'affranchir de scikit-learn, limite les itérations NMF à 30 et parallélise la prédiction LightGBM avec `num_threads=4`.

Tous les modules s'exécutent sur CPU uniquement, sans accès Internet, et sont conçus pour être réutilisables dans des environnements variés.